

Automatiser les tests d'attaque + rapport (version simple)

Sujet 3 — Adaptation minimale de votre labo existant

Guide pratique simplifié

Juillet 2026

Table des matières

1	Idée générale (en 3 phrases)	2
2	Étape 1 — Activer SSH sur Kali	3
3	Étape 2 — Créer une clé SSH sur Windows	3
4	Étape 3 — Copier la clé sur Ubuntu et Kali	3
5	Étape 4 — Autoriser sudo sans mot de passe sur Ubuntu	3
6	Étape 5 — Ajouter quelques règles Auditd sur Ubuntu	3
7	Étape 6 — Ajouter des règles Wazuh (pour reconnaître ces événements)	4
8	Étape 7 — Installer les 3 bibliothèques Python	5
9	Étape 8 — Le script complet : bas_lab.py	5
10	Étape 9 — Lancer le test	8
11	En cas de problème	8

1 Idée générale (en 3 phrases)

Vous avez déjà tout : Ubuntu (victime), Kali (attaquant), Wazuh sur Windows. On ajoute **un seul script Python** (`bas_lab.py`) qui va, tour à tour : se connecter en SSH à Kali/Ubuntu pour lancer une attaque, regarder si Wazuh a généré une alerte correspondante, et à la fin écrire un **rapport PDF** avec le résultat (détecté / non détecté) pour chaque attaque. Rien d'autre à installer que 3 petites bibliothèques Python.

Votre dossier de travail (déjà existant) : `C:\Users\ROG STRIX G17\Desktop\Projet-SIEM-IA\wazuh-docker\single-node`

C'est **dans ce même dossier** qu'on ajoute le nouveau script, à côté de `ia_detection.py`.

2 Étape 1 — Activer SSH sur Kali

Sur la VM **Kali** :

```
sudo systemctl enable --now ssh
```

3 Étape 2 — Créer une clé SSH sur Windows

Dans PowerShell, sur **Windows** :

```
ssh-keygen -t ed25519 -f "$env:UserProfile\.ssh\id_bas_lab" -N ''
```

Appuyez sur Entrée à chaque question (pas de phrase de passe).

4 Étape 3 — Copier la clé sur Ubuntu et Kali

```
type $env:UserProfile\.ssh\id_bas_lab.pub | ssh Emsi@192.168.119.140 "mkdir -p ~/.ssh  
↵ && cat >> ~/.ssh/authorized_keys"  
type $env:UserProfile\.ssh\id_bas_lab.pub | ssh kali@192.168.119.137 "mkdir -p ~/.ssh  
↵ && cat >> ~/.ssh/authorized_keys"
```

(Le mot de passe vous sera demandé une seule fois pour chaque machine.)

Test — ça ne doit **plus** demander de mot de passe :

```
ssh -i $env:UserProfile\.ssh\id_bas_lab Emsi@192.168.119.140 "hostname"  
ssh -i $env:UserProfile\.ssh\id_bas_lab kali@192.168.119.137 "hostname"
```

5 Étape 4 — Autoriser sudo sans mot de passe sur Ubuntu

Sur la VM **Ubuntu**, pour que le script puisse exécuter des commandes sudo sans bloquer :

```
echo "Emsi ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/090-bas-lab  
sudo chmod 440 /etc/sudoers.d/090-bas-lab
```

Uniquement acceptable parce que c'est une VM de test isolée.

6 Étape 5 — Ajouter quelques règles Auditd sur Ubuntu

Toujours sur **Ubuntu** :

```
sudo tee -a /etc/audit/rules.d/audit.rules > /dev/null << 'EOF'  
-w /usr/sbin/useradd -p x -k user_creation
```

```
-w /etc/hosts -p wa -k hosts_changes
-w /usr/bin/nc -p x -k suspicious_tool_nc
-w /bin/bash -p x -k shell_execution
EOF
sudo systemctl restart auditd
```

7 Étape 6 — Ajouter des règles Wazuh (pour reconnaître ces événements)

Sur **Windows**, créez le fichier `local_rules.xml` dans le dossier `single-node` :

```
<group name="local,">
  <rule id="100100" level="7">
    <if_group>audit_command</if_group>
    <field name="audit.key">user_creation</field>
    <description>Creation suspecte d'un compte utilisateur (T1136)</description>
  </rule>
  <rule id="100101" level="7">
    <if_group>audit_command</if_group>
    <field name="audit.key">hosts_changes</field>
    <description>Modification suspecte de /etc/hosts (T1565)</description>
  </rule>
  <rule id="100102" level="7">
    <if_group>audit_command</if_group>
    <field name="audit.key">suspicious_tool_nc</field>
    <description>Utilisation reseau suspecte (T1571)</description>
  </rule>
  <rule id="100103" level="5">
    <if_group>audit_command</if_group>
    <field name="audit.key">shell_execution</field>
    <match>atomic_test</match>
    <description>Commande suspecte executee (T1059)</description>
  </rule>
</group>
```

Puis, dans PowerShell, dans le dossier `single-node` :

```
docker cp local_rules.xml
↪ single-node-wazuh.manager-1:/var/ossec/etc/rules/local_rules.xml
docker exec single-node-wazuh.manager-1 /var/ossec/bin/wazuh-control restart
```

8 Étape 7 — Installer les 3 bibliothèques Python

Vous avez déjà un environnement virtuel dans ce dossier (visible dans votre `dir /S`, dossier `venv`). Activez-le et installez :

```
cd "C:\Users\ROG STRIX G17\Desktop\Projet-SIEM-IA\wazuh-docker\single-node"
.\venv\Scripts\Activate.ps1
pip install paramiko fpdf2
```

9 Étape 8 — Le script complet : `bas_lab.py`

Créez ce fichier dans le même dossier `single-node` (à côté de `ia_detection.py`). **Tout est dans ce seul fichier** — modifiez juste les deux lignes `ALERTS_PATH` et `KALI_USER` en haut si besoin.

```
"""bas_lab.py - Lance 6 scenarios d'attaque, verifie leur detection par Wazuh,
et genere un rapport PDF. Un seul fichier, pas de configuration ailleurs."""

import json, os, time
from datetime import datetime
import paramiko
from fpdf import FPDF

# --- A ADAPTER SI BESOIN ---
ALERTS_PATH = r"C:\Users\ROG STRIX
↳ G17\Desktop\Projet-SIEM-IA\wazuh-docker\single-node\wazuh_logs\alerts\alerts.json"
SSH_KEY = os.path.expanduser(r"~\.ssh\id_bas_lab")
UBUNTU = {"host": "192.168.119.140", "user": "Emsi"}
KALI = {"host": "192.168.119.137", "user": "kali"}

# --- LES 6 SCENARIOS ---
SCENARIOS = [
    {
        "id": "SCN-01", "name": "Scan reseau (Nmap)", "mitre": "T1595/T1046",
        "target": KALI, "command": "nmap -A 192.168.119.140",
        "wait": 20, "keywords": ["nmap", "rootcheck", "syslog"], "min_level": 3,
    },
    {
        "id": "SCN-02", "name": "Force brute SSH (Hydra)", "mitre": "T1110.001",
        "target": KALI,
        "command": "hydra -l Emsi -P /usr/share/wordlists/fasttrack.txt
↳ ssh://192.168.119.140 -t 64",
        "wait": 30, "keywords": ["authentication_failed", "sshd"], "min_level": 10,
    },
    {
        "id": "SCN-03", "name": "Commande suspecte", "mitre": "T1059.004",
        "target": UBUNTU,
        "command": "echo 'aWQ7IHdob2FtaTsgZWNoYBhdG9taWNfdGVzdCA+IC90bXAwdC50eHQ=' |
↳ base64 -d | bash",
        "wait": 15, "keywords": ["shell_execution", "atomic_test"], "min_level": 3,
    },
    {
        "id": "SCN-04", "name": "Creation compte utilisateur", "mitre": "T1136.001",
        "target": UBUNTU,
        "command": "sudo useradd -m -s /bin/bash attacker_test && echo
↳ 'attacker_test:Passw0rd123!' | sudo chpasswd",
    },
]
```

```

        "wait": 15, "keywords": ["user_creation", "useradd"], "min_level": 3,
    },
    {
        "id": "SCN-05", "name": "Modification fichier critique (/etc/hosts)", "mitre":
        ↪ "T1565.001",
        "target": UBUNTU,
        "command": "sudo bash -c 'echo \"6.6.6.6 malicious-c2.local\" >> /etc/hosts'",
        "wait": 15, "keywords": ["hosts_changes", "etc/hosts"], "min_level": 3,
    },
    {
        "id": "SCN-06", "name": "Connexion reseau non autorisee", "mitre": "T1571",
        "target": UBUNTU, "command": "timeout 5 nc -zv 192.168.119.137 4444 || true",
        "wait": 15, "keywords": ["suspicious_tool_nc"], "min_level": 3,
    },
]

def run_ssh(target, command):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(target["host"], username=target["user"], key_filename=SSH_KEY,
    ↪ timeout=15)
    _, stdout, stderr = client.exec_command(command, timeout=90)
    stdout.channel.recv_exit_status()
    client.close()

def file_size():
    return os.path.getsize(ALERTS_PATH) if os.path.exists(ALERTS_PATH) else 0

def read_new_alerts(offset):
    alerts = []
    if not os.path.exists(ALERTS_PATH):
        return alerts
    with open(ALERTS_PATH, "r", encoding="utf-8", errors="ignore") as f:
        f.seek(offset)
        for line in f:
            line = line.strip()
            if line:
                try:
                    alerts.append(json.loads(line))
                except json.JSONDecodeError:
                    pass
    return alerts

def is_detected(alerts, keywords, min_level):
    for a in alerts:
        text = json.dumps(a).lower()
        if any(k.lower() in text for k in keywords):
            if a.get("rule", {}).get("level", 0) >= min_level:
                return True
    return False

def run_scenario(sc):
    print(f"\n=== {sc['id']} - {sc['name']} ===")
    offset = file_size()

```

```

start = datetime.now()
run_ssh(sc["target"], sc["command"])
print(f"[+] Commande envoyée. Attente {sc['wait']}s...")
time.sleep(sc["wait"])
alerts = read_new_alerts(offset)
detected = is_detected(alerts, sc["keywords"], sc["min_level"])
print("[DETECTE]" if detected else "[NON DETECTE]")
return {
    "id": sc["id"], "name": sc["name"], "mitre": sc["mitre"],
    "command": sc["command"], "start": start.strftime("%Y-%m-%d %H:%M:%S"),
    "detected": detected,
}

def make_pdf(results, path):
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Helvetica", "B", 14)
    pdf.cell(0, 10, "Rapport de validation - Laboratoire BAS", ln=True, align="C")
    pdf.set_font("Helvetica", "", 9)
    pdf.cell(0, 8, datetime.now().strftime("Genere le %d/%m/%Y a %H:%M"), ln=True,
    ↪ align="C")
    pdf.ln(5)

    total = len(results)
    ok = sum(r["detected"] for r in results)
    pdf.set_font("Helvetica", "B", 11)
    pdf.cell(0, 8, f"Resultat global : {ok}/{total} scenarios detectes", ln=True)
    pdf.ln(3)

    for r in results:
        status = "DETECTE" if r["detected"] else "NON DETECTE"
        pdf.set_font("Helvetica", "B", 10)
        pdf.cell(0, 7, f"{r['id']} - {r['name']} [{status}]", ln=True)
        pdf.set_font("Helvetica", "", 9)
        pdf.multi_cell(0, 5, f"MITRE : {r['mitre']}\nCommande : {r['command']}\nHeure :
    ↪ {r['start']}")
        pdf.ln(2)

    pdf.output(path)

if __name__ == "__main__":
    results = [run_scenario(sc) for sc in SCENARIOS]
    os.makedirs("results", exist_ok=True)
    ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    out = f"results/rapport_{ts}.pdf"
    make_pdf(results, out)
    ok = sum(r["detected"] for r in results)
    print(f"\n=== Termine : {ok}/{len(results)} detectes ===")
    print(f"Rapport : {out}")

```

10 Étape 9 — Lancer le test

```
cd "C:\Users\ROG STRIX G17\Desktop\Projet-SIEM-IA\wazuh-docker\single-node"  
.\venv\Scripts\Activate.ps1  
python bas_lab.py
```

Le rapport PDF apparaît dans le dossier `results\`, à côté du script.

11 En cas de problème

Problème	Solution
Ça demande un mot de passe SSH sudo: password required Toujours "NON DETECTE" pour SCN-03 à 06	Refaites l'étape 3 (copie de clé) Refaites l'étape 4 (sudoers) Vérifiez <code>sudo auditctl -l</code> sur Ubuntu (étape 5), et que <code>local_rules.xml</code> est bien copié (étape 6)
Erreur <code>ModuleNotFoundError</code>	Le venv n'est pas activé, ou <code>pip install</code> pas fait (étape 7)